

halfedge：属性系统设计文档

src/property.rs

halfedge 库

2026-07-04

目录

1	模块定位	2
2	核心数据结构	2
2.1	PropertyStore<T>：单类型属性存储	2
2.2	idx 提取辅助函数	2
2.3	ErasedProperty trait（私有）	3
2.4	PropertyHandle<T>：类型化句柄	3
2.5	MeshProperties：属性容器	3
3	CRUD 方法	4
4	类型擦除流程	4
4.1	set 写入流程	4
4.2	get 读取流程	4
5	删除回调	5
6	应用：加权拉普拉斯平滑	5
6.1	公式	5
6.2	实现要点	6
7	复杂度分析	6
8	单元测试覆盖	6
9	设计取舍	7
10	使用示例	8
10.1	基础：注册并读写顶点属性	8
10.2	类型擦除：同容器并存多种类型	8
10.3	newtype：同底层类型多属性	8
10.4	半边 / 面属性：缓存边长与面法向	8

1 模块定位

`property.rs` 实现 OpenMesh 风格的属性系统，允许用户为顶点/半边/面附加**任意类型**的自定义数据。设计目标：

- 不依赖 `trait` 泛型，使用 `Any + TypeId` 实现类型擦除；
- 同一容器内可并存多种类型（`f64` / `String` / 自定义 `newtype` 等）；
- 与 `MeshStorage` 解耦，作为独立外部结构体 `MeshProperties` 使用；
- 删除网格元素时通过包装函数同步清理属性。

2 核心数据结构

2.1 `PropertyStore<T>`：单类型属性存储

```
1 pub struct PropertyStore<T> {
2     data: HashMap<usize, T>, // key = slotmap key      64      ffi (idx + version)
3 }
```

键的选取：使用 `slotmap key` 的完整 64 位 `as_ffi()` 值（`idx + version`）作为 `usize` 键，而非仅低 32 位 `idx`。原因：

- 同一元素的 `idx + version` 组合在其生命周期内唯一不变；
- `slot` 被复用时 `version` 递增，新元素得到与旧元素不同的键，旧属性条目**自动成为孤儿**（永远不会被新元素匹配到），避免了「`slot` 复用导致新元素静默继承旧属性」的隐患；
- 仍便于 `ErasedProperty::remove_by_index` 在不知类型时批量删除。

孤儿条目：删除元素后未调用 `remove_*_all_props` 时，旧属性条目不会消失，但因 `key` 中的 `version` 已过时，新元素永远不会命中它，故**无正确性问题**，仅占用少量内存。如需立即释放，可调用 `PropertyStore::clear()` 或 `remove_*_all_props`（见 §5）。

平台假设：依赖 `usize` 为 64 位（64-bit 平台）。32-bit 平台上 `as_ffi()` `as usize` 会截断高 32 位 `version`，回退到旧行为（需手动清理）。本库目标平台为 64-bit，不做特殊处理。

2.2 `idx` 提取辅助函数

`slotmap 1.1.1` 的 `KeyData` 字段为私有，公开 API 仅 `as_ffi()`。通过 `Key trait` 的 `data()` 方法取出 `KeyData`，再调 `as_ffi()` 取完整 64 位编码（`idx + version`）作为键：

```
1 #[inline]
2 fn idx_of<K: Key>(id: K) -> usize {
3     //      64      ffi      idx + version
4     //      64-bit      usize = u64
```

```

5 id.data().as_ffl() as usize
6 }

```

as_ffl() 编码格式 (slotmap 源码):

$$\text{as_ffl}() = \underbrace{(\text{version} \ll 32)}_{\text{高 32 位}} \mid \underbrace{\text{idx}}_{\text{低 32 位}}$$

对比旧设计 (仅取低 32 位 idx):

	旧设计 (仅 idx)	新设计 (idx + version)
键值	ffi & 0xFFFF_FFFF	ffi (完整 64 位)
slot 复用行为	新元素继承旧属性 (危险)	新元素得到新键, 旧条目成孤儿 (安全)
删除后清理	必须 立即调用	可延迟 (仅内存占用问题)
平台依赖	无	需 64-bit usize

2.3 ErasedProperty trait (私有)

为支持「不知类型时批量删除」, 扩展标准库 Any:

```

1 trait ErasedProperty {
2     fn as_any(&self) -> &dyn Any;
3     fn as_any_mut(&mut self) -> &mut dyn Any;
4     fn remove_by_index(&mut self, idx: usize);
5     fn clear_all(&mut self);
6 }

```

as_any 与 as_any_mut 用于 downcast 回具体 PropertyStore<T>; remove_by_index 让容器在不知 T 的情况下批量删除某 ID 的所有属性。

2.4 PropertyHandle<T>: 类型化句柄

```

1 pub struct PropertyHandle<T> {
2     _marker: PhantomData<T>,
3 }

```

零大小类型, 仅用于编译期类型推断, 是 Copy 的。属性按 TypeId 索引, 句柄本身不携带数据, 仅让 get/set/remove 方法无需显式标注泛型参数:

```

1 let h: PropertyHandle<f64> = props.add_vertex_prop();
2 props.set_vertex_prop(h, v, 1.5); // T    h        f64
3 let w = *props.get_vertex_prop(h, v).unwrap();

```

2.5 MeshProperties: 属性容器

```

1 pub struct MeshProperties {
2     vertex_props: HashMap<TypeId, Box<dyn ErasedProperty>>,
3     halfedge_props: HashMap<TypeId, Box<dyn ErasedProperty>>,

```

```

4   face_props:      HashMap<TypeId, Box<dyn ErasedProperty>>,
5 }

```

三类属性分别存储在独立 HashMap 中, 按 `TypeId::of::<T>()` 区分类型。每个类型 `T`: 'static 在每类中只能注册一个属性。

3 CRUD 方法

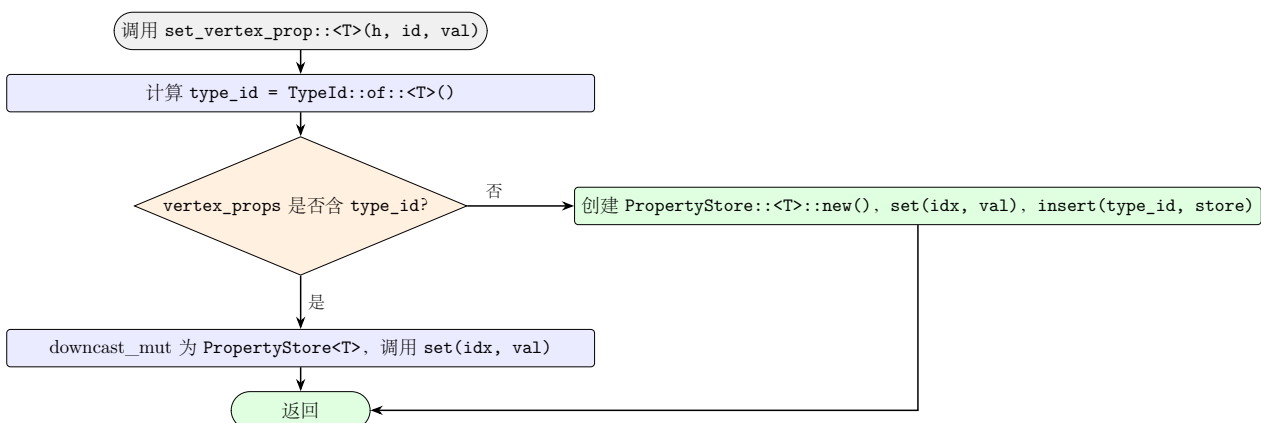
每类元素 (vertex / halfedge / face) 提供 5 个方法: `add` / `get` / `get_mut` / `set` / `remove`, 加上 `remove_*_all_props` 批量删除和 `clear_*_props` 全清。以 vertex 为例:

方法	签名	语义
<code>add_vertex_prop::<T></code>	<code>() -> PropertyHandle<T></code>	注册类型 T
<code>get_vertex_prop::<T></code>	<code>(h, id) -> Option<&T></code>	读取
<code>get_vertex_prop_mut::<T></code>	<code>(h, id) -> Option<&mut T></code>	可变读
<code>set_vertex_prop::<T></code>	<code>(h, id, val)</code>	写入 (自动注册)
<code>remove_vertex_prop::<T></code>	<code>(h, id) -> Option<T></code>	删除单类型
<code>remove_vertex_all_props</code>	<code>(id)</code>	删除所有类型
<code>has_vertex_prop::<T></code>	<code>() -> bool</code>	是否注册
<code>clear_vertex_props</code>	<code>()</code>	清空但保留类型槽

自动注册: `set_vertex_prop` 在属性未注册时自动创建 `PropertyStore<T>` 并插入, 简化用户代码。

4 类型擦除流程

4.1 set 写入流程



4.2 get 读取流程

```

1 pub fn get_vertex_prop<T: 'static>(
2     &self,
3     _handle: PropertyHandle<T>,
4     id: VertexId,
5 ) -> Option<&T> {
6     let entry = self.vertex_props.get(&TypeId::of::<T>())?;
7     let store = entry.as_any().downcast_ref::<PropertyStore<T>>()?;
8     store.get(idx_of(id))
9 }

```

三步：(1) 按 `TypeId` 查表；(2) `downcast` 回具体 `PropertyStore<T>`；(3) 按 `idx` 查值。任一环节失败返回 `None`（属性未注册 / 类型不匹配 / 该 ID 未设值）。

5 删除回调

`MeshProperties` 与 `MeshStorage` 解耦，删除网格元素时不会自动清理对应属性。提供三个包装函数同步清理：

```

1 pub fn remove_vertex_with_props(
2     mesh: &mut MeshStorage,
3     props: &mut MeshProperties,
4     id: VertexId,
5 ) -> Option<Vertex> {
6     let result = mesh.remove_vertex(id);
7     props.remove_vertex_all_props(id); // idx
8     result
9 }

```

`remove_vertex_all_props` 通过 `ErasedProperty::remove_by_index` 在不知具体类型的情况下批量删除：

```

1 pub fn remove_vertex_all_props(&mut self, id: VertexId) {
2     let idx = idx_of(id);
3     for store in self.vertex_props.values_mut() {
4         store.remove_by_index(idx);
5     }
6 }

```

6 应用：加权拉普拉斯平滑

利用顶点属性 `f64` 存储权重，执行**加权**拉普拉斯平滑：

6.1 公式

等权拉普拉斯（`geometry.rs` 中已实现）：

$$\vec{p}'_v = \frac{1}{|N(v)|} \sum_{u \in N(v)} \vec{p}_u$$

加权拉普拉斯（用属性权重 w_u ）：

$$\vec{p}'_v = \frac{\sum_{u \in N(v)} w_u \cdot \vec{p}_u}{\sum_{u \in N(v)} w_u}$$

权重 w_u 大的邻居对平滑位置贡献更大，平滑位置偏向高权重邻居。

6.2 实现要点

```
1 let w: PropertyHandle<f64> = props.add_vertex_prop();
2 for v in mesh.vertex_ids() {
3     props.set_vertex_prop(w, v, 1.0);
4 }
5 //
6 props.set_vertex_prop(w, neighbor, 10.0);
7
8 //
9 let mut sum_w = 0.0;
10 let mut sum_p = [0.0; 3];
11 for u in VertexAdjacentVerts::new(&mesh, v) {
12     let w_u = *props.get_vertex_prop(w, u).unwrap_or(&1.0);
13     let p = mesh.get_vertex(u).unwrap().position;
14     for k in 0..3 { sum_p[k] += w_u * p[k]; }
15     sum_w += w_u;
16 }
17 let weighted_avg = [sum_p[0] / sum_w, sum_p[1] / sum_w, sum_p[2] / sum_w];
```

7 复杂度分析

操作	时间复杂度	空间复杂度
add*_prop（注册类型）	$O(1)$	$O(1)$ （空 store）
get*_prop（读取）	$O(1)$	$O(1)$
set*_prop（写入）	$O(1)$ 均摊	$O(1)$ per entry
remove*_prop（删单类型）	$O(1)$	$O(1)$
remove*_all_props（删全类型）	$O(K)$, K = 已注册类型数	$O(1)$
clear*_props（清空）	$O(K + N)$, N = 总条目数	$O(1)$

8 单元测试覆盖

src/property.rs 末尾共 16 个测试，分四组：

测试名	覆盖点
<i>PropertyStore</i> 基础	
property_store_basic_crud	set / get / get_mut / remove / contains / len
property_store_iter	迭代 (idx, &T) 对
property_store_clear	clear 后 is_empty
类型擦除	
add_and_get_vertex_prop	注册 + 读写多顶点
get_vertex_prop_mut_works	可变引用就地修改
remove_vertex_prop	删除单类型 + 重复删除返回 None
remove_vertex_all_props_clears_all_types	删除顶点时清理所有类型属性
different_types_do_not_interfere	f64 / i32 / String 互不干扰
vertex_halfedge_face_props_are_independent	三类属性独立存储
自动注册与边缘情况	
get_unregistered_prop_returns_none	未注册类型读取返回 None
set_auto_registers_prop	未注册时 set 自动注册
halfedge_and_face_props_work	半边 / 面属性 CRUD + 批量删
clear_all_vertex_props	清空数据但保留类型槽
句柄与包装函数	
remove_vertex_with_props_wrapper	包装函数同步清理属性
property_handle_is_copy_and_debug	句柄 Copy + Debug
newtype_wrapper_for_multiple_same_type_props	newtype 区分同底层类型属性

cargo test 全模块共 **371** 个用例，全部通过 (0 失败 / 0 警告)。

9 设计取舍

- 为何用 Box<dyn ErasedProperty> 而非 Box<dyn Any>?

标准库 Any 不提供 remove_by_index，无法在不知 T 的情况下批量删除。ErasedProperty trait 扩展了此能力，使 remove_*_all_props 无需为每个已注册类型显式调用 remove。

- 为何属性键用完整 as_fffi() 而非仅 idx?

早期版本仅取低 32 位 idx 作键，slot 复用时新元素会**静默继承**旧元素的属性数据(MeshStorage 与 MeshProperties 解耦，用户调用 mesh.remove_vertex() 不会触发属性清理)，这是潜在的语义陷阱。改用完整 64 位 as_fffi() (idx + version) 作键后，slot 复用时 version 递增，新元素永远命中不到旧条目，旧条目成为孤儿（仅占少量内存，无正确性问题）。代价是依赖 64-bit usize，但本库目标平台已满足。

- 为何 MeshProperties 是外部结构体而非 MeshStorage 字段?

解耦让 MeshStorage 保持纯粹「存储 + 句柄有效性」职责，不绑定具体属性类型。用户可按需创建 0 个或多个 MeshProperties 实例（如分别存放几何属性与渲染属性）。

- 为何 PropertyHandle<T> 是零大小类型?

属性按 TypeId 索引，句柄仅用于编译期类型推断。零大小 Copy 类型可自由复制，无运行时开销。

- 每类型只能注册一个属性?

`TypeId` 作键意味着同类型只能存一份。需要多个同底层类型属性时, 用 `newtype` 包装: `struct Weight(f64); struct Temperature(f64);` 生成不同 `TypeId`, 互不冲突。

10 使用示例

10.1 基础: 注册并读写顶点属性

```
1 let mesh = build_icosphere(1);
2 let mut props = MeshProperties::new();
3
4 let w: PropertyHandle<f64> = props.add_vertex_prop();
5 for v in mesh.vertex_ids() {
6     props.set_vertex_prop(w, v, 1.0);
7 }
8 let v0 = mesh.vertex_ids().next().unwrap();
9 assert_eq!(props.get_vertex_prop(w, v0), Some(&1.0));
```

10.2 类型擦除: 同容器并存多种类型

```
1 let label: PropertyHandle<i32> = props.add_vertex_prop();
2 let name: PropertyHandle<String> = props.add_vertex_prop();
3 let flag: PropertyHandle<bool> = props.add_vertex_prop();
4 //                vertex_prop_type_count() == 4                f64
```

10.3 newtype: 同底层类型多属性

```
1 #[derive(Debug, PartialEq)]
2 struct Weight(f64);
3 #[derive(Debug, PartialEq)]
4 struct Temperature(f64);
5
6 let hw: PropertyHandle<Weight> = props.add_vertex_prop();
7 let ht: PropertyHandle<Temperature> = props.add_vertex_prop();
8 props.set_vertex_prop(hw, v, Weight(1.0));
9 props.set_vertex_prop(ht, v, Temperature(25.5));
```

10.4 半边 / 面属性: 缓存边长与面法向

```
1 let edge_len: PropertyHandle<f64> = props.add_halfedge_prop();
2 let normal: PropertyHandle<Vec3> = props.add_face_prop();
3
4 for he in mesh.halfedge_ids() {
5     let h = mesh.get_halfedge(he).unwrap();
6     let tip = h.vertex;
7     let origin = mesh.get_halfedge(h.twin.unwrap()).unwrap().vertex;
```



```
8     let l = dist(mesh.get_vertex(tip).unwrap().position,  
9                 mesh.get_vertex(origin).unwrap().position);  
10    props.set_halfedge_prop(edge_len, he, l);  
11 }
```

10.5 删除元素同步清理

```
1 use halfedge::property::remove_vertex_with_props;  
2  
3 let removed = remove_vertex_with_props(&mut mesh, &mut props, vids[0]);  
4 assert!(removed.is_some());  
5 assert!(mesh.contains_vertex(vids[0]));  
6 assert_eq!(props.get_vertex_prop(w, vids[0]), None); //
```